

Vyvážené stromy

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

18.09.2026

CC BY-NC-SA 4.0

Motivace

Cena degenerovaného stromu

Minule jsme skončili u **degenerovaného stromu**

– vložili jsme 1, 2, 3, 4, 5 a vznikl nám spojový seznam

Kolik porovnání potřebujeme pro vyhledání v milionu seřazeně vložených prvků?

- Degenerovaný strom: **1 000 000** porovnání
- Vyvážený strom: **20** porovnání

Stejná datová struktura, stejná data – jen jiné **pořadí vkládání**

Cíl vyvažování

Jak byste zajistili, že strom nikdy nezdegeneruje?

Po každém vložení a mazání si musíme **hlídat tvar stromu** a případně ho **opravit**

- Potřebujeme **kritérium**, které pozná nevyvážený strom
- Potřebujeme **operaci**, která strom opraví a zachová vlastnost BST

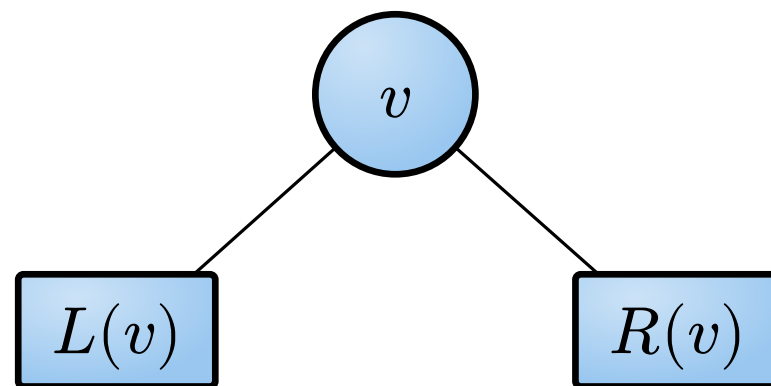
Obojí musíme zvládnout v čase $\mathcal{O}(\log n)$, jinak si nepomůžeme

Zavedení pojmů

Značení potomků a podstromů

Abychom mohli vyváženost popsat přesně, zavedeme si značení:

- $l(v)$, $r(v)$ – **levý** a **pravý potomek**
- $p(v)$ – **otec** (rodič) uzlu v
- $L(v)$, $R(v)$ – **levý** a **pravý podstrom**
- $h(T)$ – **výška** stromu T
- $|T|$ – **počet uzlů** stromu T



Prázdný podstrom

Jakou výšku má strom, který neexistuje?

Domluvíme se, že prázdný podstrom má výšku $h = -1$

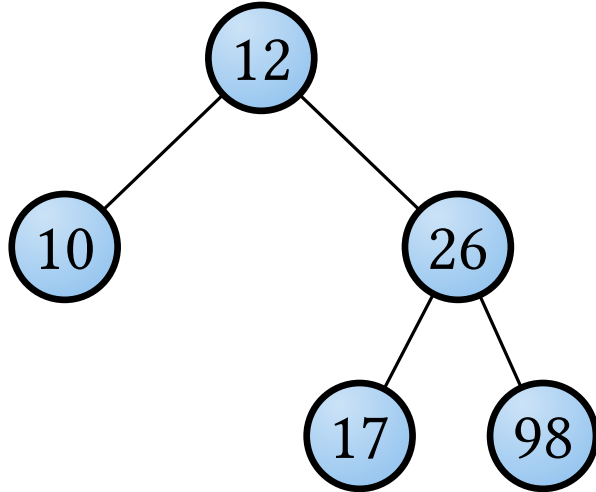
Proč právě -1 ? Aby list (uzel bez potomků) vycházel jako **vyvážený**:

$$h(R(v)) - h(L(v)) = (-1) - (-1) = 0$$

Zároveň platí $h(v) = 1 + \max(h(L(v)), h(R(v)))$

Pro listy: $1 + (-1) = 0$

Procvičení značení



- $h(T) = 2$
- $|T| = 5$
- $|R(12)| = 3$
- $h(R(12)) = 1$
- $h(L(10)) = -1$
- $l(r(12)) = 17$

Dokonale vyvážený strom

Definice dokonalé vyváženosti

O **dokonale vyváženém stromu** mluvíme, když pro **každý** uzel v platí:

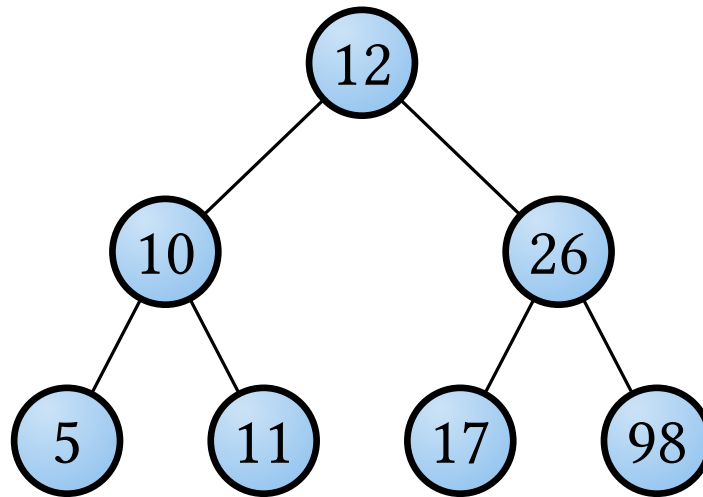
$$| |L(v)| - |R(v)| | \leq 1$$

Slovně: každý uzel má v obou podstromech (skoro) stejně **uzlů**

Takový strom má vždy jen $\lfloor \log_2 n \rfloor + 1$ hladin – méně hladin z n uzlů sestavit nelze

Definice dokonalé vyváženosti

Dokonale vyvážený strom se sedmi uzly:



Kořen má v obou podstromech 3 uzly, každý potomek po jednom

Problém dokonale vyváženého stromu

Co se stane, když do takového stromu vložíme jediný prvek?

Nový uzel zvýší $|L(v)|$ nebo $|R(v)|$ o jedničku **u každého předka** na cestě ke kořeni – pokud tam byl rozdíl 1, stane se 2 a podmínka padne

Oprava pak může znamenat přesun kořene a přestavbu obou podstromů – až $\mathcal{O}(n)$ uzlů

Operace **Insert** a **Delete** tak mají v nejhorším případě složitost $\Omega(n)$

Cílem vyhledávacích stromů byla ale **lepší než lineární** složitost

Řešení: podmínku vyváženosti mírně „uvolníme“

AVL stromy

Definice AVL stromu

Georgij **A**delson-**V**elskij a Jevgenij **L**andis (1962) navrhli podmínku, která se nedívá na **počty uzlů**, ale na **výšky** podstromů

Binární vyhledávací strom je **AVL stromem**, pokud pro každý uzel v platí:

$$| h(L(v)) - h(R(v)) | \leq 1$$

Všimněte si rozdílu: dokonalá vyváženost počítala uzly, AVL měří jen **výšku** – podstromy tak mohou mít výrazně různý počet uzlů

Faktor vyvážení

Pro každý uzel si udržujeme **faktor vyvážení** (*balance factor*) $\delta(v)$:

$$\delta(v) = h(R(v)) - h(L(v))$$

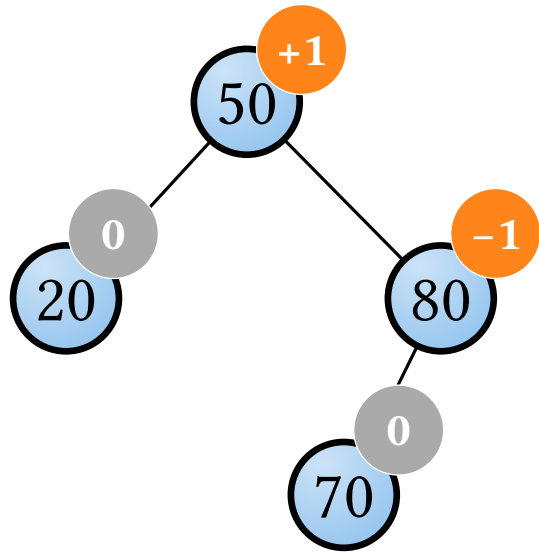
Ve správně vyváženém AVL stromě nabývá jen tří hodnot:

- $\delta(v) = 0$ – oba podstromy jsou stejně vysoké
- $\delta(v) = -1$ – levý podstrom je o jedno vyšší
- $\delta(v) = +1$ – pravý podstrom je o jedno vyšší

Jakmile se objeví $\delta(v) = \pm 2$, musíme zasáhnout

Výpočet faktoru vyvážení

Spočítejme δ pro každý uzel:

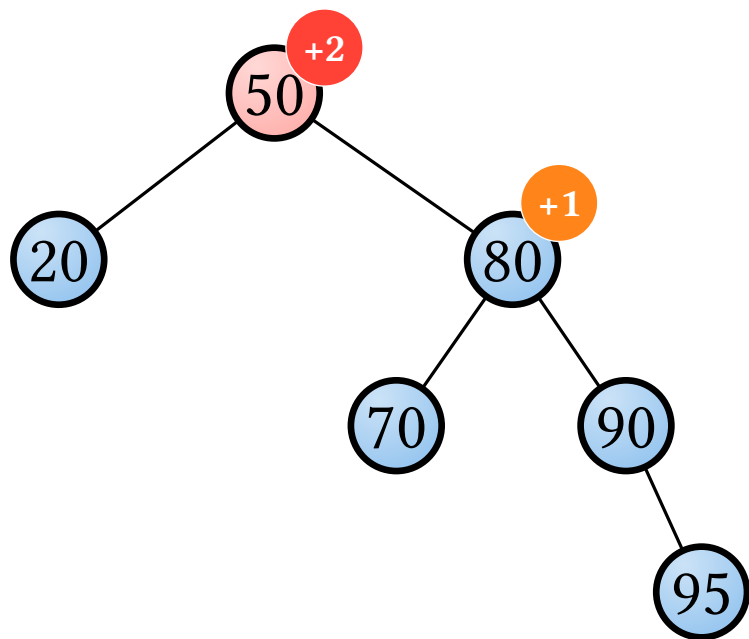


- $h(L(50)) = 0, h(R(50)) = 1$
- $\delta(50) = 1 - 0 = +1$
- $h(L(80)) = 0, h(R(80)) = -1$
- $\delta(80) = (-1) - 0 = -1$

Všechna δ jsou v rozsahu $\langle -1, +1 \rangle \rightarrow$
platný AVL strom

Výpočet faktoru vyvážení

A co tento strom?



- $h(L(50)) = 0, h(R(50)) = 2$
- $\delta(50) = 2 - 0 = +2$

Kořen podmínku **porušuje** – strom je korektní BST, ale **není** AVL

Uzel 50 je nutné **vyvážit**

Maximální výška AVL stromu

Vyváženost je uvolněná – **jak vysoký tedy AVL strom může být?**

Ptejme se obráceně: **kolik nejméně uzlů** musí mít AVL strom výšky h ?

Označme si to $N(h)$

Jeden podstrom musí mít výšku $h - 1$ (jinak by strom neměl výšku h)
a pro nejméně uzlů chceme druhý co nejnižší – AVL dovolí $h - 2$:

$$N(h) = 1 + N(h - 1) + N(h - 2)$$

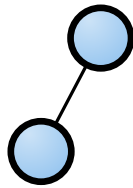
Připomíná vám to něco? Je to **stejná rekurence** jako u Fibonacciho posloupnosti z přednášky o rekurzi

Maximální výška AVL stromu

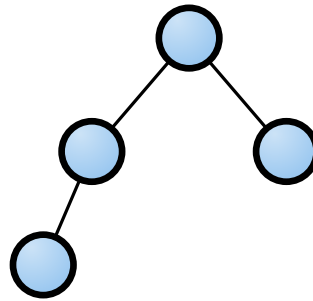
Nejtenčí možné AVL stromy výšek 0, 1, 2 a 3:



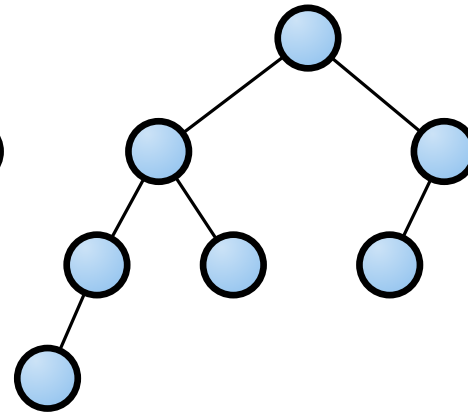
$h = 0$



$h = 1$



$h = 2$



$h = 3$

Maximální výška AVL stromu

Kolik uzlů tedy nejméně potřebujeme na jednotlivé výšky?

h	0	1	2	3	4	5	6	7	8	...
$N(h)$	1	2	4	7	12	20	33	54	88	...

Počet uzlů roste **exponenciálně** → výška jen **logaritmicky**

Z rekurence proto vyplývá horní odhad výšky AVL stromu s n uzly:

$$h \leq 1.44 \cdot \log_2 n$$

Maximální výška AVL stromu

Ověřme si to na milionu uzlů:

- Dokonale vyvážený strom: $\log_2(10^6) \approx$ výška **20**
- AVL strom v nejhorším případě: $1.44 \cdot 20 \approx$ výška **29**
- Degenerovaný strom: výška **999 999**

AVL strom je nejvýš o ~44 % vyšší než dokonale vyvážený – to je cena za rychlé vkládání

Rotace

Rotace jako nástroj

Jak strom opravíme, když v nějakém uzlu vyjde $\delta(v) = \pm 2$?

Použijeme **rotaci** – lokální přepojení uzlu, jeho potomka a jejich podstromů

Rotace musí splnit tři věci:

- zachovat **vlastnost BST**, tedy inorder pořadí hodnot
- snížit výšku vyššího podstromu (a tím vyrovnat δ)
- trvat $\mathcal{O}(1)$ – přepojíme jen **pár ukazatelů**

Rotace nemění pořadí, jen tvar!

Přehled čtyř rotací

Rotaci vybíráme podle $\delta(v)$ a δ jeho **vyššího potomka**:

$\delta(v)$	δ potomka	Případ	Rotace
+2	+1	RR	levá
-2	-1	LL	pravá
-2	+1	LR	levo-pravá
+2	-1	RL	pravo-levá

Levá rotace

Levou rotaci provádíme v uzlu z , pokud platí

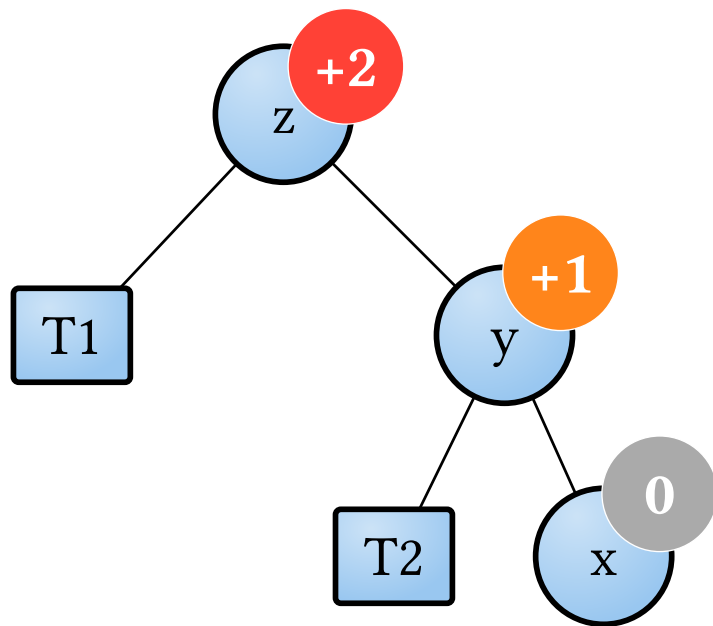
$$\delta(z) = +2 \quad \wedge \quad \delta(r(z)) = +1$$

Novým kořenem podstromu se stane pravý potomek y , uzel z se stane jeho **levým** potomkem

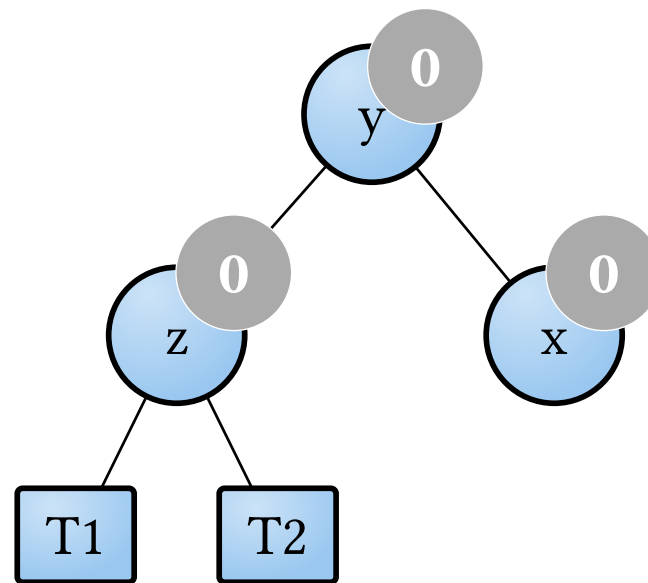
Podstrom $T2$ přitom „přesedlá“ od uzlu y k uzlu z – inorder pořadí $T1$, z , $T2$, y , x zůstává stejné

Levá rotace

Před rotací

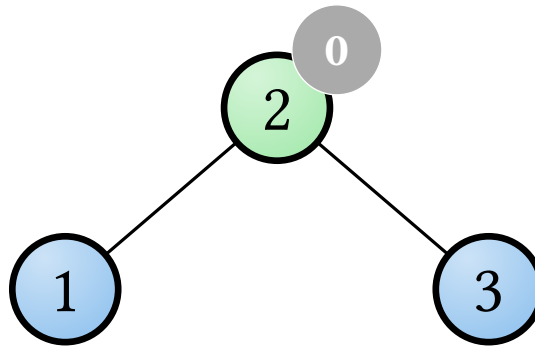


Po rotaci



Příklad na levou rotaci

Vložme postupně 1, 2, 3:



Po levé rotaci se novým kořenem stal uzel **2** → výška 1 místo 2

Pravá rotace

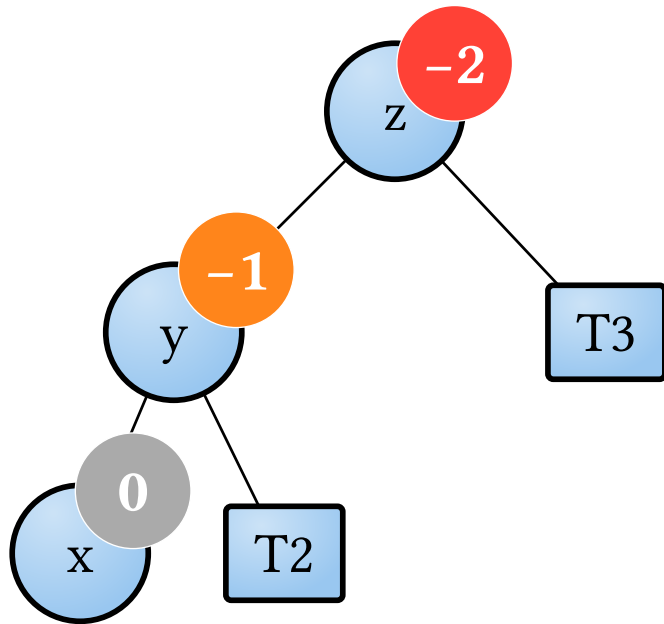
Pravá rotace je zrcadlovým obrazem levé – provádíme ji v uzlu z , pokud platí

$$\delta(z) = -2 \quad \wedge \quad \delta(l(z)) = -1$$

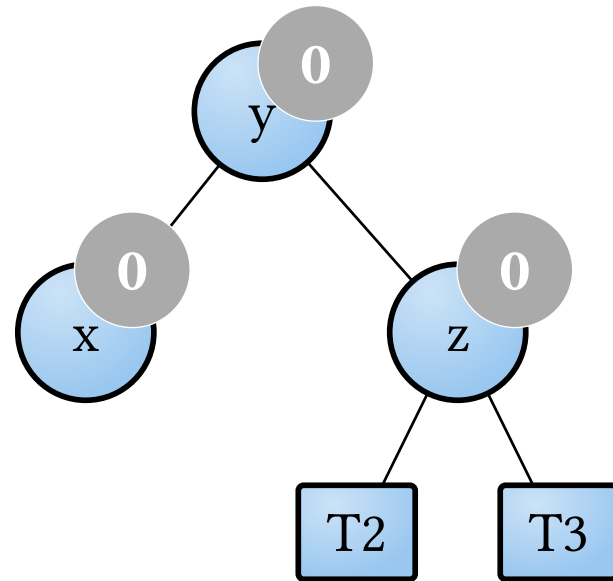
Novým kořenem podstromu se stane levý potomek y , uzel z se stane jeho **pravým** potomkem

Pravá rotace

Před rotací

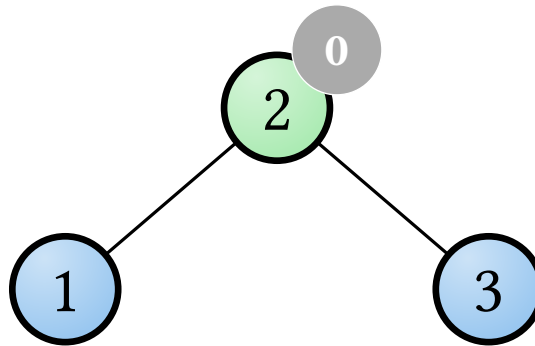


Po rotaci



Příklad na pravou rotaci

Vložme postupně 3, 2, 1:



Po pravé rotaci je kořenem uzel **2**

Selhání jednoduché rotace

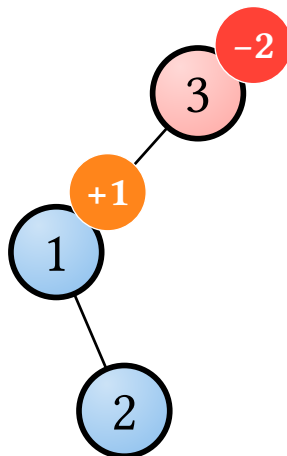
Zamyslete se: pomůže jednoduchá rotace vždy?

Vložme 3, 1, 2 – kořen 3 má $\delta = -2$, ale jeho potomek 1 je převážený **doprava**, tedy $\delta = +1$

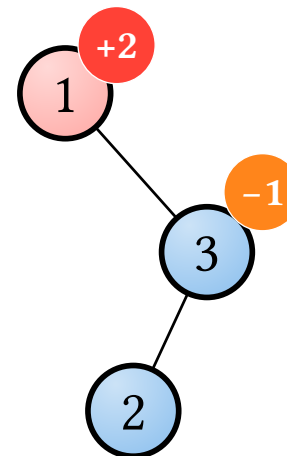
Zkusme na uzel 3 přesto použít **pravou rotaci**

Selhání jednoduché rotace

Před rotací



Po pravé rotaci



Strom se jen **překlopil** – pořád **není**
AVL!

Levo-pravá rotace

Levo-pravou rotaci provádíme v uzlu z , pokud platí

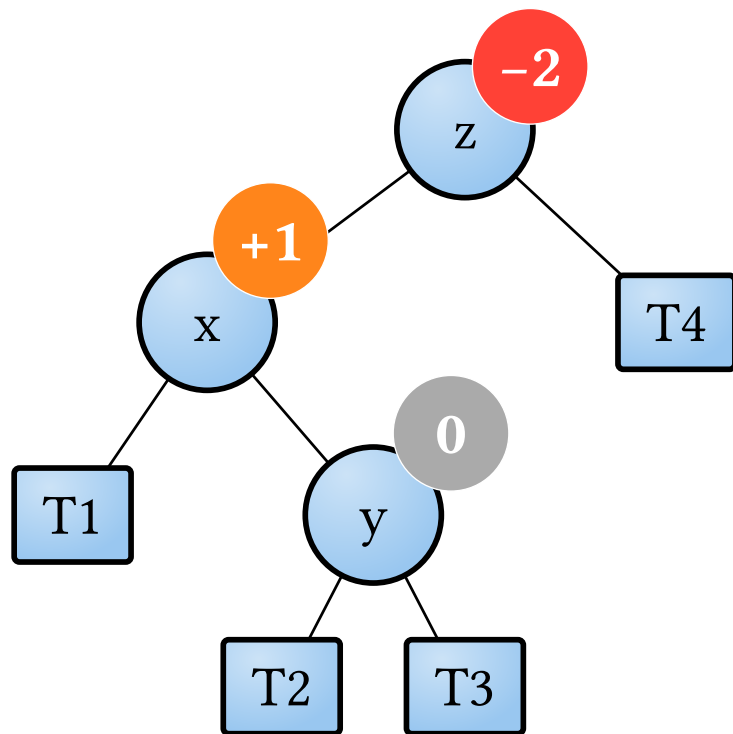
$$\delta(z) = -2 \quad \wedge \quad \delta(l(z)) = +1$$

Rotujeme **nejprve doleva** v potomkovi x a **poté doprava** v uzlu z

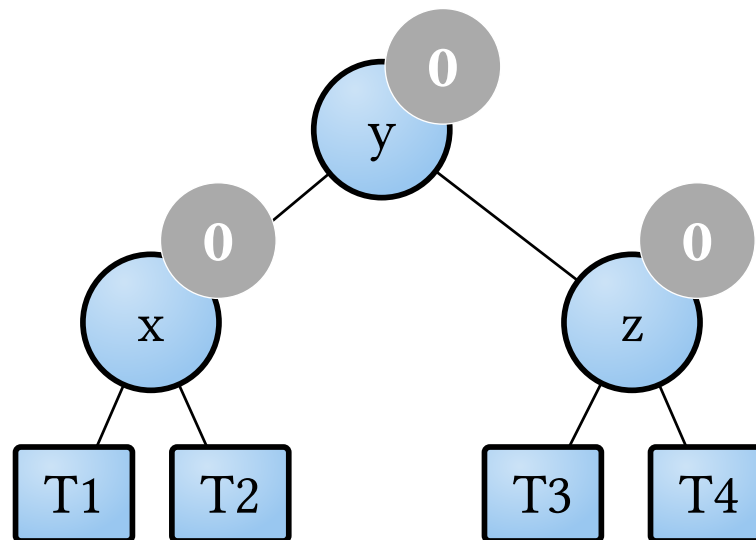
Novým kořenem podstromu je **vnouček** y , ne potomek x

Levo-pravá rotace

Před rotací

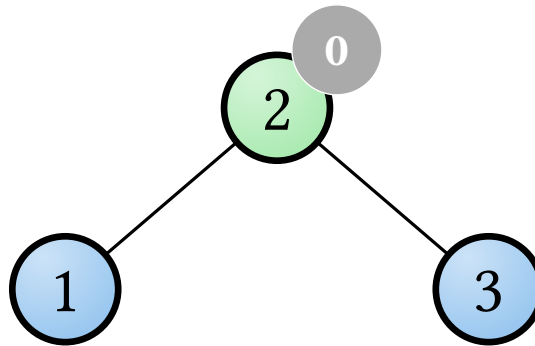


Po rotaci



Příklad na levo-pravou rotaci

Vraťme se k neúspěšné trojici 3, 1, 2:



Dvojitá rotace vyváží strom

Pravo-levá rotace

Pravo-levou rotaci provádíme v uzlu z , pokud platí

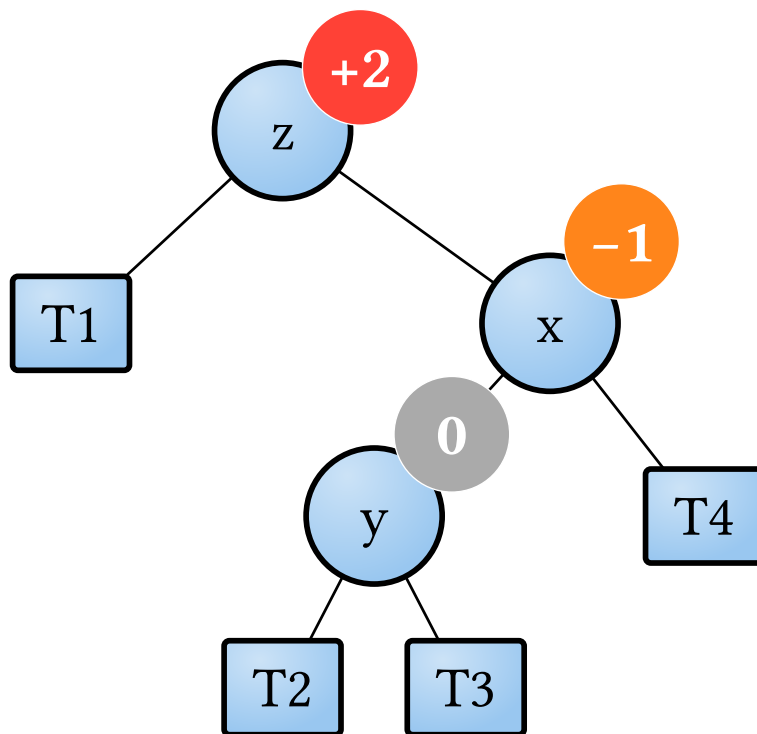
$$\delta(z) = +2 \quad \wedge \quad \delta(r(z)) = -1$$

Rotujeme **nejprve doprava** v potomkovi x a **poté doleva** v uzlu z

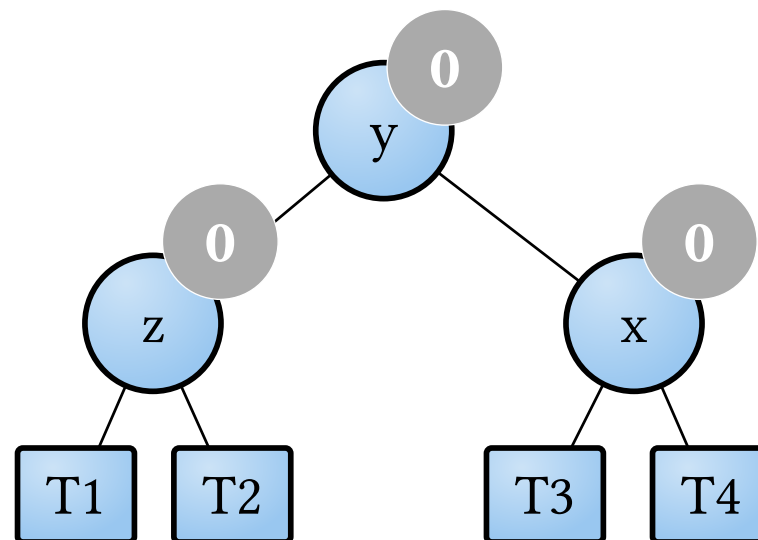
I zde je novým kořenem podstromu **vnouček** y

Pravo-levá rotace

Před rotací

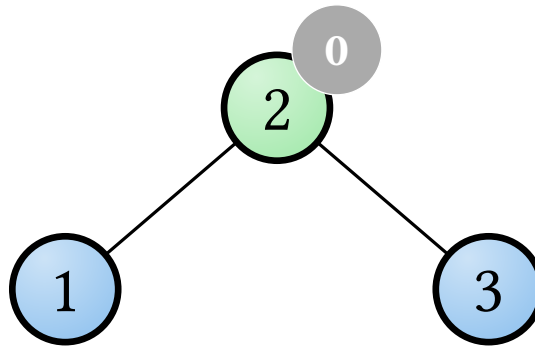


Po rotaci



Příklad na pravo-levou rotaci

Vložme 1, 3, 2:



Všechny čtyři případy končí stejně – **vyváženým podstromem**

Vkládání a mazání

Vkládání do AVL stromu

Algoritmus:

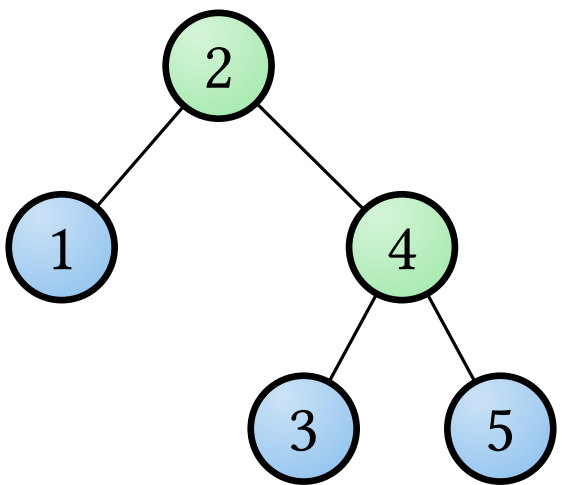
1. Vlož nový uzel **stejně jako do BST**
2. Vracej se cestou zpět ke kořeni a **aktualizuj výšky**
3. V **prvním** uzlu s $|\delta(v)| = 2$ proved' příslušnou rotaci
4. **Hotovo** – žádná další rotace není potřeba

Důležité: jedna (jednoduchá nebo dvojitá) rotace při vkládání **vždy stačí** – sníží výšku podstromu zpět na původní hodnotu, takže výš už se nic nerozváží

Výšky ale aktualizujeme **až ke kořeni**

Vyvážené vkládání 1 až 5

Zkusme znovu vložit 1, 2, 3, 4, 5 – tentokrát s vyvažováním:



Výsledek: výška **2** místo **4** – za pět vložení jen **dvě** rotace

Mazání z AVL stromu

Algoritmus:

1. Smaž uzel **stejně jako v BST** – list, jeden potomek, nebo in-order následník
2. Vracej se ke kořeni, aktualizuj výšky a vyvažuj každý uzel na cestě

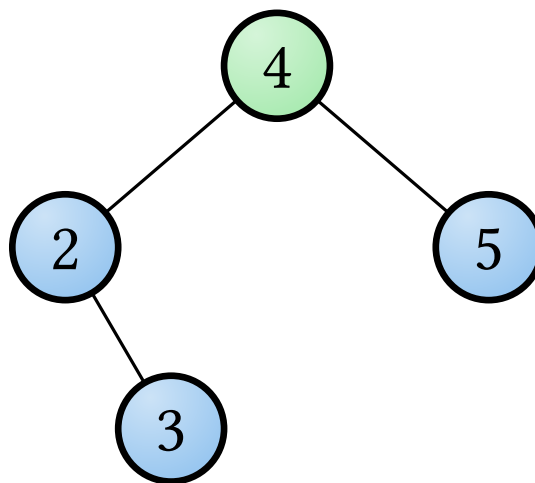
Pozor: jedna rotace zde **nemusí stačit** – rotace podstrom zkrátí a může rozvážit uzel výš

V nejhorším případě až $\mathcal{O}(\log n)$ rotací – jednu na každé hladině

Navíc: u potomka může vyjít $\delta = 0$ (při vkládání to nenastane) → volíme **jednoduchou** rotaci

Příklad mazání s rotací

Smažme z právě postaveného stromu uzel **1**:



Strom je opět AVL – právě zde se uplatnil případ δ potomka rovného
nule

Složitost a praxe

Složitost operací

Operace	BST (nejhorší)	AVL strom
Vyhledávání	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$
Vložení	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$
Smazání	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$

Jedna rotace stojí $\mathcal{O}(1)$ a na cestě ke kořeni jich je nejvýš $\mathcal{O}(\log n)$

Klíčové: AVL zaručuje i **nejhorší případ** – degenerace už nastat nemůže

AVL a červeno-černé stromy

AVL není jediný samovyvažovací strom:

- **AVL** – výška $\leq 1.44 \log_2 n \rightarrow$ rychlejší **vyhledávání**, více rotací
- **Červeno-černý strom** – výška $\leq 2 \log_2 n \rightarrow$ méně rotací, rychlejší zápisy

Fakt: v knihovnách vyhrály červeno-černé stromy – `std::map` v C++, `TreeMap` v Javě, `rbtree` v jádře Linuxu

Databáze používají **B-stromy** optimalizované na čtení bloků z disku

Vyzkoušejte si to sami

Rotace se nejlépe pochopí **rukou na papíře** – zkuste do prázdného AVL stromu vložit postupně 5, 50, 10, 15, 20, 45, 30 a sledujte, kdy nastane která rotace

Interaktivní vizualizace: visualgo.net/en/avl

A ve volných chvílích se podívejte na `rbtree` v jádře Linuxu – je to stejná myšlenka, jen s barvami místo výšek

Shrnutí

- **AVL strom** nahrazuje drahou dokonalou vyváženost (počty uzlů) podmínkou na **výšky**: $|\delta(v)| \leq 1$
- Čtyři rotace (levá, pravá, levo-pravá, pravo-levá) trvají $\mathcal{O}(1)$ a zachovají inorder pořadí
- Vkládání potřebuje **nejvýš jednu** rotaci, mazání až $\mathcal{O}(\log n)$ rotací
- Výška AVL stromu je vždy $\leq 1.44 \log_2 n \rightarrow$ všechny operace v $\mathcal{O}(\log n)$
- V praxi se častěji používají **červeno-černé stromy** – volnější vyvážení, méně rotací